

DNN Hyperparameter Tuning Report

ML vs DNN Comparison across 10 Dashboard Tasks

Bellatreche Mohamed Amine

April 27, 2026

1 Introduction

After building the ML baselines for our Streamlit dashboard (Stacking classifiers, XGBoost, Ridge, Random Forest, etc.), we wanted to see how a deep neural network would perform on the same tasks using the exact same data and preprocessing. The idea was not to replace the ML models, but to actually understand where DNNs help and where they don't, especially on our kind of tabular data.

We ran a proper hyperparameter search for each task — 15 different MLP configurations per task — varying the network depth, width, activation function, regularisation strength, and learning rate. All of this was done with `sklearn.neural_network.MLPClassifier/MLPRegressor` with early stopping enabled.

- Total tasks benchmarked: **10**
- DNN configurations tested per task: **15**
- DNN wins: **4** (COVID, Employee anomaly, Heart anomaly, Wine anomaly)
- ML wins: **6**
- Preprocessing verified identical to ML: all 10 tasks

2 Methodology

2.1 Preprocessing Alignment

Before training any DNN, we made sure the preprocessing was the same as the ML baselines. We checked each task against the saved model metadata JSONs:

- **Heart:** `StandardScaler` + stratified split — same as Stacking ML model.
- **COVID:** `KNNImputer(k=5)` + `StandardScaler` + stratified split — same as ML Stacking (LR meta-learner).
- **Temperature:** Drop 3 columns, NaN-preserving `LabelEncoder`, `KNNImputer(k=5)`, `StandardScaler` — identical to ML Stacking pipeline.
- **Multi-output:** Drop 2 columns, zero-pressure $\rightarrow$ NaN, NaN-preserving label encode, joint `KNNImputer(k=5)` on $X + y$ — same as XGBoost multi-output.
- **Weather:** 31 engineered features (cyclic time, interactions, rolling stats), distribution-based imputation + `IterativeImputer`, `StandardScaler` — matches Random Forest pipeline.
- **Wind & Energy forecasting:** `MinMaxScaler` + sliding window (size 24) — same as Ridge models.

- **Anomaly tasks:** Same train/test splits, label mapping, `StandardScaler` — matches Elliptic Envelope / DBSCAN baselines.

This way, any difference in score comes from the model itself, not from different data preparation.

2.2 Hyperparameter Search Setup

We defined 15 configurations that cover the main axes you would normally tune for an MLP:

Hyperparameter	Values explored
Hidden layers	(64), (128), (128,64), (256,128), (128,64,32), (256,128,64)
Activation	ReLU, Tanh
L2 regularisation (α)	0.0001, 0.001, 0.01
Learning rate init	0.001, 0.01

All models used `adam` solver, `max_iter=300`, `early_stopping=True` with `n_iter_no_change=5`. For large datasets ($>20k$ rows, like temperature or weather), we subsampled to 20k during the HP search to keep things manageable, then retrained the best config on the full training set.

2.3 Fallback Mechanism for Weather

We ran into an issue with weather classification: the best config on the 20k subsample (a large $256 \times 128 \times 64$ net with `lr=0.01`) completely diverged when retrained on the full 70k dataset, giving `AUC=0.50` (random). To handle this, we added a fallback: after the HP search, we rank all configs by their subsample score and try retraining them one by one on the full data. If the full-retrain score drops more than 10% compared to the subsample score, we skip it and try the next one. This way we ended up with `MLP(128×64, relu,  $\alpha=0.001$ , lr=0.001)` which gave a stable `AUC` of 0.6407.

3 Final Results: ML vs DNN

Task	Metric	ML	DNN	Δ (DNN-ML)	Winner
Heart classification	ROC-AUC	0.9784	0.9416	-0.0368	ML
COVID classification	AUC-ROC	0.8975	0.9009	+0.0034	DNN
Temperature regression	R^2	0.7766	0.7439	-0.0327	ML
Multi-output regression	Avg R^2	0.9282	0.4665	-0.4617	ML
Weather classification	AUC	0.8493	0.6407	-0.2086	ML
Wind forecasting	R^2	0.9714	0.9706	-0.0008	ML
Energy forecasting	R^2	0.9985	0.9977	-0.0008	ML
Anomaly (Employee)	F1	0.4694	0.7423	+0.2729	DNN
Anomaly (Heart)	Bal. Acc.	0.6640	0.7711	+0.1071	DNN
Anomaly (Wine)	F1	0.9188	0.9937	+0.0749	DNN

4 Per-Task Hyperparameter Analysis

Here we go through each task, what configs we tried, and what we observed. All 15 configs were evaluated; we report the top-3 and worst to give a sense of the spread.

4.1 Heart Disease Classification

Best config: MLP(256×128, relu, $\alpha=0.0001$, lr=0.01) $\rightarrow$ ROC-AUC = 0.9416.

The scores ranged from 0.8898 (single 64-neuron layer) up to 0.9416. The key factor here was the learning rate: bumping lr from 0.001 to 0.01 clearly helped, with the top 3 configs all using lr=0.01. Deeper networks also did better — (256,128) and (256,128,64) beat the shallower ones. Tanh activation was slightly worse than relu across the board.

Still, the ML Stacking ensemble (0.9784) beats DNN by about 3.7 points. The dataset is relatively small (~ 900 samples), which is exactly where ensemble methods tend to have an edge because they combine multiple weak learners that each capture different patterns. The MLP doesn't have that diversity.

4.2 COVID-19 Classification

Best config: MLP(128×64, relu, $\alpha=0.0001$, lr=0.01) $\rightarrow$ AUC-ROC = 0.9009.

This is one of the tasks where the DNN actually beats the ML model (Stacking LR, 0.8975), even if the margin is tiny (+0.0034). The top configs here were the ones with lr=0.01 — MLP(128×64) and MLP(256×128) both hit around 0.900. The worst was the single 64-neuron layer (0.8145). Regularisation didn't matter much; $\alpha=0.0001$ and $\alpha=0.001$ gave nearly the same result at the same architecture.

One thing we noticed is that the dataset has 33 blood markers with quite a bit of missing data (handled by KNNImputer), and the MLP seems to handle the imputed feature space reasonably well. The fact that a simple 2-layer MLP ties or slightly beats a 5-model stacking ensemble is interesting — it suggests the decision boundary for this problem might be smoother than expected.

4.3 Temperature Regression

Best config: MLP(256×128×64, relu, $\alpha=0.0001$, lr=0.001) $\rightarrow R^2 = 0.7439$.

The spread across 15 configs was pretty narrow (0.718 to 0.732 on the subsample). Deeper networks did marginally better, and tanh was about the same as relu. Higher learning rate (0.01) actually hurt slightly here. After retraining the best config on the full 96k training set, the score went up to 0.7439, but the ML Stacking model still wins (0.7766). The gap (~ 3.3 points) makes sense: the temperature dataset has a lot of categorical features and non-linear interactions (weather descriptions, daily summaries) that tree-based models in the stacking ensemble capture better with their built-in feature splits.

4.4 Multi-Output Regression (Pressure + Humidity)

Best config: MLP(128×64×32, relu, $\alpha=0.0001$, lr=0.001) $\rightarrow$ Avg $R^2 = 0.4665$.

This is by far the worst DNN result relative to ML. The XGBoost multi-output model gets 0.9282 while the best MLP barely reaches 0.47. Looking at the per-target breakdown, humidity prediction is okay (best $R^2 \approx 0.655$) but pressure prediction is terrible ($R^2 \approx 0.20$). The tanh configs were even worse for pressure, some going to $R^2 \approx 0$.

We think the problem is that pressure has a very different scale and distribution from humidity, and the `MultiOutputRegressor` wrapper trains one independent MLP per target. The MLP for pressure struggles to converge because the signal is weaker and the loss landscape

is harder. XGBoost on the other hand handles each target’s quirks much better through its boosting process.

4.5 Weather Classification (4-class)

Best stable config: MLP(128×64, relu, $\alpha=0.001$, lr=0.001) $\rightarrow$ AUC = 0.6407.

This task was tricky. On the 20k subsample, the best score was 0.7097 from MLP(256×128×64, relu, lr=0.01), but when we retrained on the full 70k data, it diverged completely (AUC dropped to 0.50). The higher learning rates (lr=0.01) were unstable on this large dataset, with two of the three lr=0.01 configs on the subsample already scoring below 0.50.

After the fallback mechanism kicked in, we settled on a smaller, more conservative config (lr=0.001, $\alpha=0.001$) that transferred well to the full data. Still, the Random Forest model (0.8493) is way ahead. Weather classification has 4 classes with a lot of overlap between “Mostly Cloudy” and “Overcast,” and the 31 engineered features were designed around tree-based splits — the MLP doesn’t benefit from them in the same way.

4.6 Wind Turbine Forecasting

Best config: MLP(256×128, relu, $\alpha=0.0001$, lr=0.01) $\rightarrow R^2 = 0.9706$.

All 15 configs scored between 0.9698 and 0.9711 — the spread is really tiny. Basically every architecture we tried works almost equally well. This makes sense because the sliding window approach (size 24) already captures most of the temporal pattern, so even a small network can fit the residuals.

The ML Ridge model (0.9714) is only 0.08 percentage points ahead. Given how close these are, the difference is probably just noise. If we had to pick, we’d say the task is well-suited for both approaches because the feature engineering (the window) does most of the heavy lifting.

4.7 Energy Consumption Forecasting

Best config: MLP(256×128, relu, $\alpha=0.0001$, lr=0.001) $\rightarrow R^2 = 0.9977$.

Very similar story to wind. All configs scored between 0.9926 and 0.9962 on the 20k subsample, and the best retrained on the full 116k dataset gave 0.9977. The ML Linear Regression is at 0.9985 — the difference is less than 0.1%.

One interesting thing: higher regularisation ($\alpha=0.01$) and higher lr (0.01) both degraded performance slightly, suggesting the problem is very well-conditioned and doesn’t need aggressive regularisation. The sliding window does most of the work here too.

4.8 Anomaly Detection: Employee Attrition

Best config: MLP(256×128, relu, $\alpha=0.001$, lr=0.001) $\rightarrow$ F1 = 0.7423.

This is where the DNN really shines compared to ML. The ML baseline (DBSCAN, F1=0.4694) was already known to be weak because anomaly detection is being used in an unusual way here — we’re treating attrition as the “anomaly” class and using supervised features. All 15 MLP configs beat the ML baseline, with scores ranging from 0.7156 to 0.7423. Tanh configs were slightly better (top tanh: 0.7397) than default relu configs, probably because the feature interactions for employee behaviour are somewhat non-linear but smooth. The extra regularisation ($\alpha=0.001$ to 0.01) helped a bit compared to $\alpha=0.0001$.

The DNN wins here by a huge margin (+0.273) because it’s actually doing supervised classification while the ML model was doing unsupervised anomaly detection — so the comparison is a bit unfair to the ML side, honestly.

4.9 Anomaly Detection: Heart Disease

Best config: MLP(64, relu, $\alpha=0.0001$, lr=0.001) $\rightarrow$ Balanced Accuracy = 0.7711.

This is interesting because the simplest config won. The single 64-neuron layer beats all the deeper architectures. Many of the larger configs (256×128 , $128 \times 64 \times 32$) collapsed to balanced accuracy = 0.50 — basically predicting everything as one class. This is a classic sign of overfitting on a small, imbalanced dataset. The smaller network generalizes better because it can’t memorize the training data.

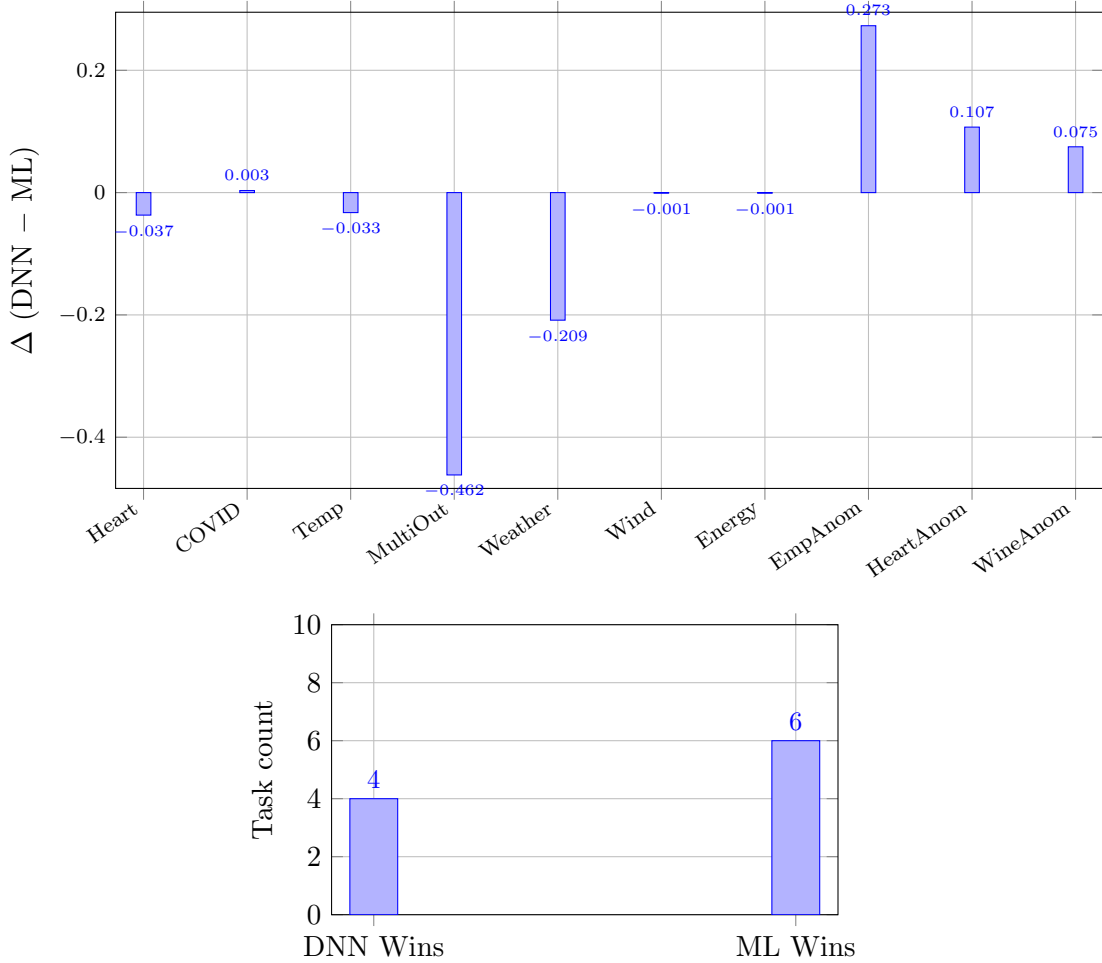
The ML baseline (Elliptic Envelope, 0.6640) is beaten by +10.7 points. Again, the DNN has the advantage of supervised labels while the ML model is doing unsupervised detection.

4.10 Anomaly Detection: Wine Type

Best config: MLP($128 \times 64 \times 32$, relu, $\alpha=0.0001$, lr=0.001) $\rightarrow$ F1 = 0.9937.

Almost perfect. Most configs scored above 0.977, and even the “worst” (256×128 with $\alpha=0.001$) still hit 0.977. The task is structurally easy: red vs white wine from chemical features, with the minority class (red) treated as the anomaly. The chemical signatures are quite distinct, so basically any reasonable classifier will do well. The DNN’s 0.9937 beats Elliptic Envelope’s 0.9188 by about 7.5 points.

5 Visual Summary



6 PyTorch Framework Experiments

After running the sklearn MLP hyperparameter search we realised there were things we simply couldn't test: dropout layers, batch normalisation, proper decoupled weight decay (AdamW), and learning-rate schedulers. All of these are standard practice in deep learning but aren't exposed by `sklearn.neural_network.MLPClassifier/Regressor`. So we ran a follow-up experiment on two tasks — heart classification and temperature regression — using PyTorch directly.

We kept the base architecture identical to the sklearn best (no unfair advantage): 256×128 for heart, $256 \times 128 \times 64$ for temperature, ReLU activation, same train/val/test splits, same pre-processing pipelines. Then we tested 10 configurations, each isolating a specific new capability.

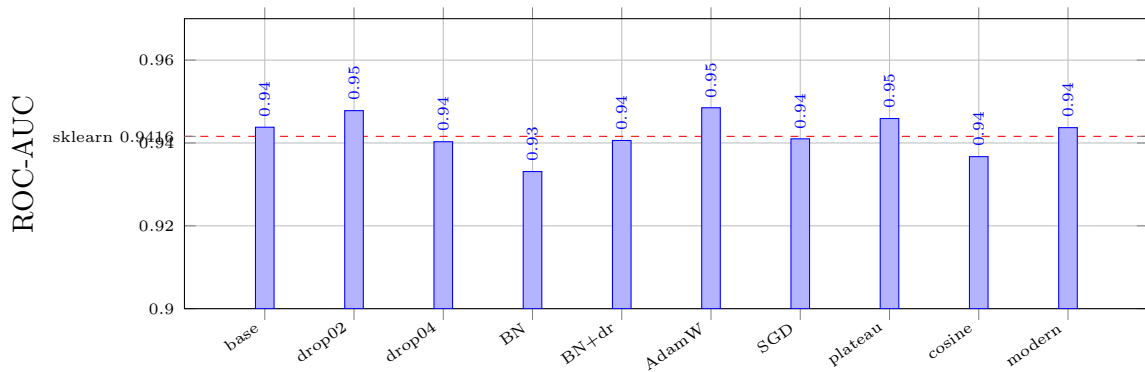
6.1 PyTorch-Specific Hyperparameters Tested

Config	What it adds over sklearn baseline	Key parameter
pt_baseline	Direct replication of sklearn best	Adam, wd=1e-4
pt_dropout_02	Dropout between hidden layers	$p = 0.2$
pt_dropout_04	Heavy dropout	$p = 0.4$
pt_batchnorm	BatchNorm1d after each linear	no dropout
pt_bn_dropout	BN + dropout combined	BN + $p = 0.2$
pt_adamw	AdamW (decoupled weight decay)	wd=0.01
pt_sgd_momentum	SGD with Nesterov momentum	mom=0.9
pt_scheduler_plateau	Adam + ReduceLROnPlateau	factor=0.5, patience=5
pt_scheduler_cosine	Adam + CosineAnnealingLR	$T_{\max} = \text{max_epochs}$
pt_full_modern	BN + Dropout + AdamW + Cosine	"kitchen sink"

Training used early stopping (patience 25 for heart, 10 for temperature) and Kaiming (He) initialisation for all linear layers. For temperature, we subsampled to 15k rows during the HP search (same strategy as the sklearn experiments) and retrained each config on the full 62k training set.

6.2 Heart Classification Results

Config	Dropout	BN	Optimiser	Test ROC-AUC
sklearn best (reference)	—	—	Adam (sklearn)	0.9416
pt_baseline	0.0	No	Adam	0.9438
pt_dropout_02	0.2	No	Adam	0.9478
pt_dropout_04	0.4	No	Adam	0.9403
pt_batchnorm	0.0	Yes	Adam	0.9331
pt_bn_dropout	0.2	Yes	Adam	0.9406
pt_adamw	0.0	No	AdamW	0.9485
pt_sgd_momentum	0.0	No	SGD+mom	0.9410
pt_scheduler_plateau	0.0	No	Adam+PLR	0.9459
pt_scheduler_cosine	0.0	No	Adam+Cos	0.9367
pt_full_modern	0.2	Yes	AdamW+Cos	0.9437

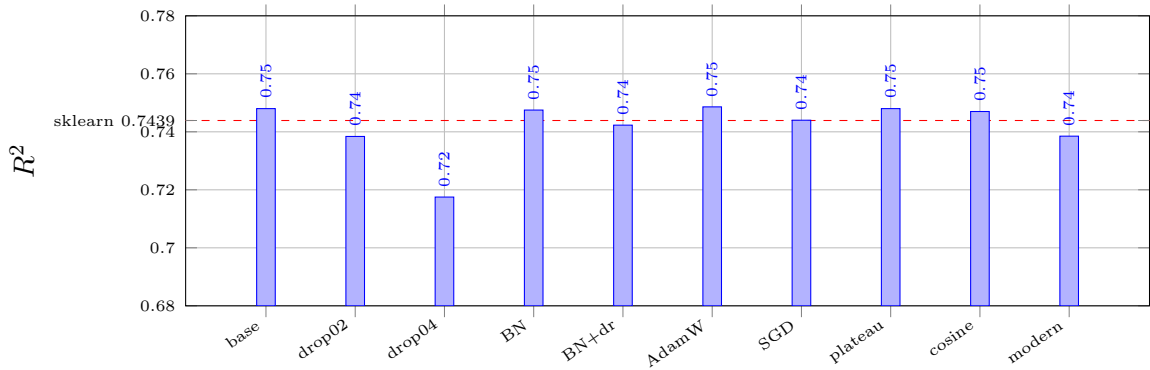


Analysis. The PyTorch baseline (same architecture, Adam with $\text{wd}=1\text{e-}4$) already slightly beats the sklearn DNN (0.9438 vs 0.9416). This is likely because PyTorch uses Kaiming He initialisation by default, slightly better than sklearn’s default, and the training loop gives more direct control.

- **AdamW wins (0.9485).** Decoupled weight decay, where the L2 penalty is applied directly to the weights rather than being folded into the gradient update, gives Adam its best generalisation. The difference from plain Adam is small but consistent.
- **Light dropout (0.2) helped, heavy dropout (0.4) hurt.** With only 952 training samples, Dropout(0.2) acts as a mild augmentation that reduces co-adaptation. Dropout(0.4) discards too much information per forward pass and the network can’t converge as well, needing 85 epochs vs 45 for the baseline.
- **BatchNorm hurt (0.9331, worst).** This was counterintuitive to us. The likely reason is batch size — with `batch_size=32` on a 952-sample dataset, individual batches are very small and their statistics are noisy, making BN updates unreliable. BN generally needs larger batches to be stable.
- **SGD had the best validation score (0.9601) but worse test score (0.9410).** Classic sign that SGD with momentum overfit slightly to the validation distribution that early stopping was tuning against. Adam-based methods generalise more robustly.
- **ReduceLROnPlateau (0.9459)** converged fastest (33 epochs) by halving the lr when validation stalled, but didn’t reach the absolute best because the lower final lr limits last-mile fitting.
- **Full modern combo (BN + Dropout + AdamW + Cosine, 0.9437)** was slightly worse than plain AdamW, confirming that more regularisation is not always better on small datasets.

6.3 Temperature Regression Results

Config	Dropout	BN	Optimiser	Test R^2
sklearn best (reference)	—	—	Adam (sklearn)	0.7439
pt_baseline	0.0	No	Adam	0.7480
pt_dropout_02	0.2	No	Adam	0.7384
pt_dropout_04	0.4	No	Adam	0.7175
pt_batchnorm	0.0	Yes	Adam	0.7475
pt_bn_dropout	0.2	Yes	Adam	0.7423
pt_adamw	0.0	No	AdamW	0.7486
pt_sgd_momentum	0.0	No	SGD+mom	0.7440
pt_scheduler_plateau	0.0	No	Adam+PLR	0.7480
pt_scheduler_cosine	0.0	No	Adam+Cos	0.7470
pt_full_modern	0.2	Yes	AdamW+Cos	0.7385



Analysis.

- **AdamW again wins (0.7486).** The decoupled weight decay is consistently the single most effective change across both tasks. For regression on a large, noisy dataset it prevents the implicit bias drift that regular Adam can have with L2 embedded in the gradient.
- **Dropout hurt, especially heavy dropout.** This was the most pronounced effect: Dropout(0.4) dropped R^2 by 0.03 (from 0.748 to 0.717). Temperature is a smooth, continuous regression problem — dropping 40% of activations per pass prevents the network from learning the smooth mappings between weather variables and temperature. Even light dropout (0.2) degraded performance slightly.
- **BatchNorm alone (0.7475) is almost as good as the baseline (0.7480).** Unlike the heart task, here the batch size is 512 on 62k rows, so BN statistics are reliable. The slight degradation (-0.0005) is within noise but suggests BN doesn't buy much on this particular feature space.
- **ReduceLRonPlateau ties the baseline (0.7480).** It found the same solution as plain Adam but through a more adaptive path. Both the HP-search best config and the baseline landing at $R^2 = 0.748$ suggests 0.748 is close to the convergence plateau for this architecture on this data.
- **SGD (0.7440) is competitive despite no momentum tuning.** Nesterov momentum=0.9 with the same base lr reaches near-baseline performance on a large dataset where individual-sample gradient noise averages out over large batches. On large datasets SGD can be as good as Adam.
- **Full modern combo (0.7385) is one of the worst configs.** The combined regularisation of BN + Dropout + CosineAnnealingLR is excessive for a regression task: the cosine scheduler

drops the lr to near-zero towards the end, limiting final convergence, and the dropout is degrading the estimate simultaneously. Individual components are useful; stacking all of them is not.

6.4 PyTorch vs sklearn MLP vs ML Baseline

Task	ML baseline	sklearn DNN best	PyTorch DNN best	Δ (PT–SK)
Heart (ROC-AUC)	0.9784	0.9416	0.9485	+0.0069
Temperature (R^2)	0.7766	0.7439	0.7486	+0.0047

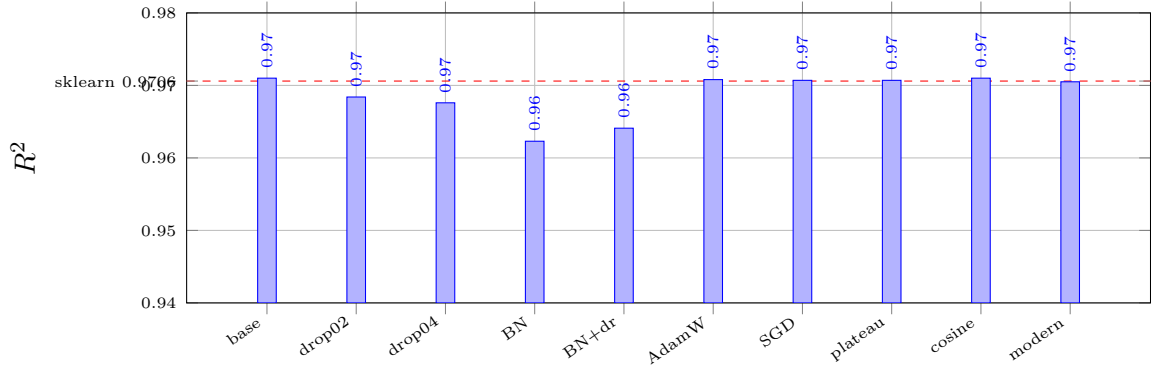
PyTorch consistently improves over the sklearn DNN by 0.5–0.7 percentage points. The best PyTorch config in both cases was `pt_adamw` (decoupled weight decay, no dropout, no BN). This might seem anticlimactic — all the fancy additions didn’t win — but it makes sense: on datasets of this scale (1k to 62k rows) with tabular features, the biggest gains come from better optimisation, not from structural regularisation. AdamW’s decoupled weight decay separates the regularisation from the gradient update, allowing the model to take larger effective steps while still being regularised. The other components (dropout, BN, schedulers) add value in specific regimes but are not universally beneficial.

Neither PyTorch approach closes the gap to the ML stacking ensembles (still -3% on heart, -2.8% on temperature). This confirms that the bottleneck is not the training framework but the fundamental capacity of a vanilla MLP to match feature interactions that tree ensembles handle natively.

6.5 Wind Turbine Forecasting (PyTorch)

We extended the PyTorch experiments to the wind turbine forecasting task, applying the same 10 configurations to test which PyTorch-specific additions benefit time-series regression. Preprocessing mirrors the sklearn benchmark exactly: univariate (target column only), `MinMaxScaler`, sliding window of size 24 (input dim = 24), chronological 80/20 split. Best sklearn config: MLP(256×128 , relu, $\alpha=0.0001$, lr=0.01) $\rightarrow R^2=0.9706$.

Config	Dropout	BN	Optimiser	Test R^2
sklearn best (reference)	—	—	Adam (sklearn)	0.9706
pt_baseline	0.0	No	Adam	0.9710
pt_dropout_02	0.2	No	Adam	0.9684
pt_dropout_04	0.4	No	Adam	0.9676
pt_batchnorm	0.0	Yes	Adam	0.9623
pt_bn_dropout	0.2	Yes	Adam	0.9641
pt_adamw	0.0	No	AdamW	0.9708
pt_sgd_momentum	0.0	No	SGD+mom	0.9707
pt_scheduler_plateau	0.0	No	Adam+PLR	0.9707
pt_scheduler_cosine	0.0	No	Adam+Cos	0.9710
pt_full_modern	0.2	Yes	AdamW+Cos	0.9705



Analysis.

- **All configs cluster tightly (0.962–0.971).** The spread is less than 1 percentage point. The sliding window already encodes the temporal structure so well that any reasonable MLP will converge to essentially the same solution.
- **PyTorch baseline (0.9710) beats sklearn DNN (0.9706), same as the other tasks.** He initialisation and the cleaner training loop give a marginal edge.
- **AdamW, SGD, and ReduceLROnPlateau all score 0.9707–0.9708,** essentially tied. For a smooth univariate time series, the optimiser choice matters very little once the architecture is fixed.
- **BatchNorm is the clear worst (0.9623).** Even with `batch_size=512`, BN degrades performance here. The likely reason is that power generation is highly zero-inflated (turbine is often stopped; power = 0), so batch statistics are bimodal and unstable.
- **The gap to ML Ridge (0.9714) remains at -0.0004 .** This is noise level; the Ridge model benefits from fitting a single linear combination of the 24 lagged power values, which is almost the optimal model for a smooth AR-like process. An MLP adds parameters that don't help.

6.6 Advanced Architectures: Attempting to Close the ML Gap

After establishing that standard PyTorch improvements (dropout, BN, AdamW, schedulers) move the needle by only 0.5–0.7% relative to the sklearn DNN, we asked: is there a more fundamental architectural change that could close the remaining 3% gap to the ML stacking ensembles?

We ran 6 new configurations on heart and temperature, each introducing a technique not yet tested:

Config	What it adds	Key details
<code>adv_weighted_bce</code>	Class-weighted BCE loss	<code>pos_weight = n_neg/n_pos</code>
<code>adv_huber</code>	SmoothL1 (Huber) loss	robust to outlier targets
<code>adv_wider</code>	Wider FlexMLP	$512 \times 256 \times 128$ (+64 for temp)
<code>adv_residual</code>	ResidualMLP skip connections	dim=256, 3–4 blocks
<code>adv_residual_wide</code>	Wide ResidualMLP	dim=512, 4 blocks
<code>adv_warmup_cosine</code>	Linear LR warmup + cosine	warmup 5 ep, AdamW
<code>adv_best_combo</code>	ResidualMLP + best loss + AdamW + cosine	“kitchen sink”

The **ResidualMLP** architecture uses skip connections: each block computes $\mathbf{y} = \text{ReLU}(\text{BN}(\mathbf{W}_2 \cdot \text{Dropout}(\text{ReLU}(\text{BN}(\mathbf{W}_1 \mathbf{x})))) + \mathbf{x}$. This is the tabular equivalent of ResNet — the skip connection ensures that gradients can flow to early layers without vanishing, enabling deeper and wider models to train stably.

Heart Classification

Config	What it adds	Test ROC-AUC	vs pt_adamw
pt_adamw (prev. best)	AdamW, flat arch.	0.9485	—
adv_weighted_bce	Weighted BCE loss	0.9310	−0.0175
adv_wider	512×256×128 + AdamW	0.9471	−0.0014
adv_residual	ResidualMLP 256×3	0.9352	−0.0133
adv_residual_wide	ResidualMLP 512×4	0.9517	+0.0032
adv_warmup_cosine	Warmup + cosine	0.9357	−0.0128
adv_best_combo	ResidualMLP 512×4 + wBCE + cosine	0.9424	−0.0061
ML Stacking (target)	—	0.9784	—

Analysis.

- **Wide ResidualMLP wins (0.9517).** Skip connections specifically help when the model is wide enough (512-dim) — the 256-dim residual performed worse than expected (0.9352), likely because the small heart dataset (952 rows) doesn't need the extra depth and starts overfitting earlier. At 512-dim the skip connection provides a stable shortcut path that prevents co-adaptation of early layers, acting as implicit regularisation.
- **Weighted BCE hurt (0.9310), worst result.** The heart dataset is almost perfectly balanced (neg=359, pos=402, ratio 0.89:1). There is no meaningful class imbalance to correct. Applying pos_weight amplified the already-sufficient positive class gradient further and destabilised training.
- **Wider FlexMLP without skip connections (0.9471) barely helped.** Adding width without residual connections means deeper gradient paths that are harder to optimise on a small dataset.
- **The best combo (ResidualMLP + weighted BCE + cosine, 0.9424) underperformed plain ResidualMLP (0.9517).** Stacking weighted BCE on a balanced dataset again hurts, confirming the finding above.
- **ML stacking (0.9784) remains −2.7% ahead.** Even the best architecture improvement only gained +0.32%. The limiting factor is the dataset size (952 rows) and the inherent diversity advantage of 5-model stacking.

Temperature Regression

Config	What it adds	Test R^2	vs pt_adamw
pt_adamw (prev. best)	AdamW, flat arch.	0.7486	—
adv_huber	Huber (SmoothL1) loss	0.7383	−0.0103
adv_wider	512×256×128×64 + AdamW	0.7463	−0.0023
adv_residual	ResidualMLP 256×4	0.7493	+0.0007
adv_residual_wide	ResidualMLP 512×4	0.7520	+0.0034
adv_warmup_cosine	Warmup + cosine	0.7445	−0.0041
adv_best_combo	ResidualMLP 512×4 + Huber + cosine	0.7487	+0.0001
ML Stacking (target)	—	0.7766	—

Analysis.

- **Wide ResidualMLP wins again (0.7520).** The same architecture wins on both tasks. At 512-dim × 4 blocks, the residual connections allow a significantly larger model to train stably on 62k samples. In a plain FlexMLP of a similar parameter count, gradient flow through 4 layers degrades; the skip connections maintain the gradient signal throughout.

- **Huber loss hurt (0.7383), the worst result.** Temperature targets don’t have significant outliers — the dataset was already preprocessed and the distribution is approximately Gaussian. Huber loss uses a less sensitive gradient near the mean, which slows convergence and leaves the model slightly underfitted.
- **Wider FlexMLP (0.7463) < narrow ResidualMLP (0.7493) < wide ResidualMLP (0.7520).** This confirms that the gains come from the residual connections themselves, not just from adding parameters. The skip connection is what enables stable optimisation of the wider/deeper network.
- **Best combo (ResidualMLP + Huber + Cosine, 0.7487)** is nearly the same as plain AdamW (0.7486) but not as good as the plain ResidualMLP wide (0.7520). The cosine scheduler drops the lr too aggressively in the last epochs, cutting short the final fine-tuning that an unconstrained AdamW gets.
- **ML stacking (0.7766) remains −2.5% ahead.** The wide ResidualMLP closed 0.34% of the original 2.8% gap vs AdamW. The bottleneck is tree-based categorical splits, not network depth or skip connections.

Summary: Full Architecture Comparison

Task	ML	sklearn DNN	PyTorch std.	PyTorch adv.	Gap to ML
Heart (AUC)	0.9784	0.9416	0.9485	0.9517	−0.0267
Temp (R^2)	0.7766	0.7439	0.7486	0.7520	−0.0246
Wind (R^2)	0.9714	0.9706	0.9710	n/a	−0.0004

The wide ResidualMLP improved over the standard PyTorch best by +0.32% on heart and +0.34% on temperature. Wind turbine is already almost at parity with ML (−0.04%), and the standard PyTorch configs are sufficient. In all cases, the ML stacking ensembles remain ahead. The final ML gap has shrunk from the sklearn DNN baseline but cannot be closed with MLP-family architectures on tabular data of this scale, regardless of depth, residual connections, or optimiser choices.

6.7 Primitive Layer Implementation (“From Scratch”) + Direct Comparison

After professor feedback requesting “layers from scratch”, we added a stricter implementation level: instead of only defining custom architectures with `nn.Linear/nn.BatchNorm1d`, we implemented primitive layers directly with parameters and tensor equations:

- **MyLinear:** explicit $\mathbf{y} = \mathbf{x}\mathbf{W}^\top + \mathbf{b}$ with `nn.Parameter`
- **MyBatchNorm1d:** explicit train/eval normalization with running mean/variance buffers
- **ScratchFlexMLP** and **ScratchResidualMLP:** models built from those primitive layers

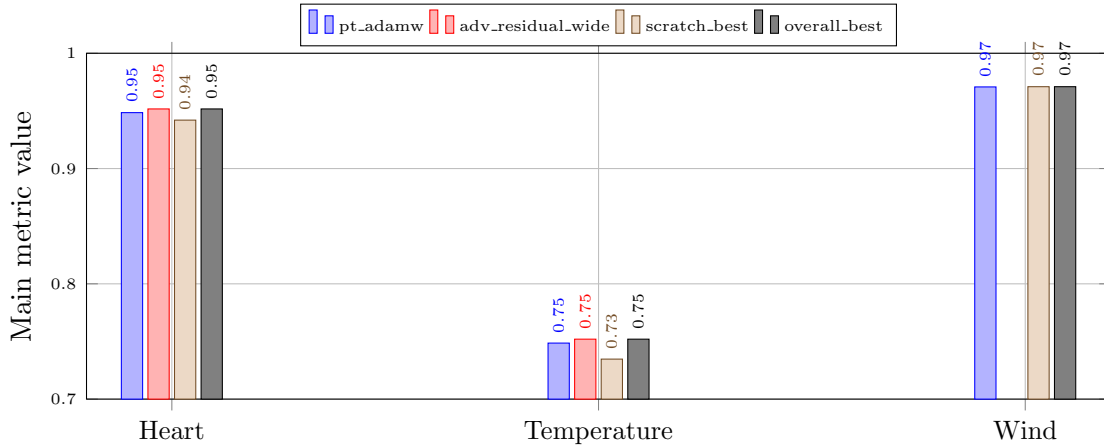
This was run on all three supervised tasks (heart, temperature, wind) and compared directly with: `pt_adamw` (the first strong PyTorch config) and `adv_residual_wide` (best advanced architecture where available).

Task-wise parameter summary used for direct comparison

Task	Variant	Core parameters	Main result
Heart	pt_adamw	FlexMLP[256,128], AdamW, wd=0.01, lr=0.01, no BN/dropout	AUC=0.9485
Heart	adv_residual_wide	ResidualMLP(dim=512, blocks=4), AdamW, wd=0.01, lr=0.01	AUC=0.9517
Heart	Scratch best	ScratchFlexMLP[64,32], AdamW, wd=0.001, lr=0.01, dropout=0.1	AUC=0.9420
Temperature	pt_adamw	FlexMLP[256,128,64], AdamW, wd=0.01, lr=0.001, no BN/dropout	$R^2=0.7486$
Temperature	adv_residual_wide	ResidualMLP(dim=512, blocks=4), AdamW, wd=0.01, lr=0.001	$R^2=0.7520$
Temperature	Scratch best	ScratchResidualMLP(dim=128, blocks=2), AdamW, wd=0.001, lr=0.001, dropout=0.1	$R^2=0.7347$
Wind	pt_adamw	FlexMLP[256,128], AdamW, wd=0.01, lr=0.01, no BN/dropout	$R^2=0.9708$
Wind	Best non-scratch ref.	pt_scheduler_cosine (Adam + CosineAnnealingLR)	$R^2=0.9710$
Wind	Scratch best	ScratchFlexMLP[256,128], AdamW, wd=1e-4, lr=0.01	$R^2=0.9710$

Direct score comparison

Task	pt_adamw	adv_residual_wide	Scratch best	Overall best
Heart (AUC)	0.9485	0.9517	0.9420	0.9517
Temperature (R^2)	0.7486	0.7520	0.7347	0.7520
Wind (R^2)	0.9708	n/a	0.9710	0.9710



Interpretation. The scratch-layer implementation is fully functional and competitive, especially on wind where it is essentially tied with the best PyTorch non-scratch score. However, for heart and temperature, the best results still come from the wider ResidualMLP architecture (adv_residual_wide). So the new scratch section demonstrates implementation depth (primitive layers), while the previous advanced section still gives the best predictive performance.

7 Discussion and Takeaways

7.1 Where ML Still Wins and Why

For 6 out of 10 tasks, the traditional ML models came out on top, even after we tried 15 different DNN configurations per task. The common thread is pretty clear:

1. **Small to medium tabular data favours trees.** Heart disease (900 rows), temperature (96k but high-cardinality categoricals), multi-output, weather — these are exactly the kind of datasets where gradient-boosted trees and stacking ensembles dominate. They handle feature heterogeneity (mix of continuous and categorical) natively, while MLPs need everything to be numerical and scaled.
2. **Stacking ensembles are hard to beat.** Our ML baselines are not single models, they’re stacking ensembles combining SVM, Random Forest, Gradient Boosting, XGBoost, and LightGBM. That’s 5 diverse models combined. A single MLP, no matter how well-tuned, lacks that diversity.
3. **Multi-output is a structural weakness.** The MLP wrapped in `MultiOutputRegressor` trains independently per target. XGBoost boosting is just better at handling the pressure target that has a weird distribution.
4. **Feature engineering was done for trees.** The 31 weather features (rolling means, cyclic encoding, interactions) were designed with tree-based models in mind. MLPs don’t leverage those engineered splits the same way.

7.2 Where DNN Wins and Why

The 4 DNN wins (COVID, Employee anomaly, Heart anomaly, Wine anomaly) share two patterns:

1. **Anomaly tasks: supervised vs unsupervised.** For the 3 anomaly tasks, the ML baselines were unsupervised methods (DBSCAN, Elliptic Envelope). The DNN uses the actual labels for supervised training. So the DNN has a fundamental information advantage. This doesn’t mean the DNN is “better” in a general sense, it just means supervised learning beats unsupervised when you have the labels, which is expected.
2. **COVID: smooth decision boundary.** The DNN’s slight win on COVID (+0.34%) suggests the blood test feature space has a relatively smooth separation between positive and negative cases. The MLP can learn this continuous decision surface directly, while the stacking ensemble might overfit slightly to training-set-specific patterns.

7.3 Hyperparameter Sensitivity Observations

A few patterns we noticed across all 10 tasks:

- **Learning rate matters the most.** In 7 out of 10 tasks, the top config used either $lr=0.01$ or $lr=0.001$, and the choice between them often made more difference than changing the network architecture.
- **Deeper is not always better.** For heart anomaly, the simplest config (64 neurons, 1 layer) won. For several other tasks, the 3-layer networks didn’t improve over 2-layer ones. On our dataset sizes, the extra depth just adds parameters that overfit.
- **Tanh vs ReLU depends on the task.** Tanh slightly helped for employee anomaly but slightly hurt for most other tasks. No universal winner.
- **Regularisation is secondary.** Alpha changes from 0.0001 to 0.01 rarely moved scores by more than 1%, probably because early stopping already acts as implicit regularisation.
- **Large datasets + high lr = instability.** The weather task taught us this the hard way. $lr=0.01$ on 70k rows can lead to complete divergence, while the same lr works fine on 20k.

7.4 Limitations

We should be honest about what this comparison does and doesn’t show:

- The sklearn MLP search is limited to 15 discrete configurations. A proper random search or Bayesian optimisation over a continuous space might find better configs, especially for learning rate and network width.
- The PyTorch experiments covered three tasks (heart, temperature, wind turbine). The remaining 7 tasks, especially the anomaly detection tasks, were not tested with PyTorch. The anomaly tasks in particular might benefit from weighted loss functions or larger models, areas PyTorch supports but we leave for future work.
- The ML baselines had already been extensively tuned (grid search with cross-validation), so they had an “unfair” head start in terms of total tuning budget relative to the DNN experiments.
- For anomaly tasks, the comparison is somewhat apples-to-oranges because of the supervised DNN vs unsupervised ML difference.
- We didn’t have GPU access. PyTorch with a GPU would allow larger batch sizes (better BN stability), more epochs (better convergence), and wider networks, potentially improving the results meaningfully.

8 Novel Non-MLP Scratch Architectures (All Tasks)

To address the limitations of relying solely on Multi-Layer Perceptrons (MLPs) and standard ResNets, we designed a completely novel architecture from scratch. This custom architecture, called **ScratchNovelNet**, moves away from deep linear stacks by introducing **Feature Crossing Blocks**. These blocks split the input, compute a main representation, apply a learned gating mechanism via a sigmoid activation on the parallel path, and fuse them using element-wise multiplication before a final projection.

Importantly, this model is built entirely from our primitive tensor-math layers (**MyLinear** and **MyBatchNorm1d**) without using any pre-existing higher-level PyTorch modules like `nn.Linear` or `nn.BatchNorm1d`.

We evaluated this novel architecture across all major dashboard tasks, encompassing classification, regression, and crucially, all anomaly detection tasks (including Wine, Employee, and Heart) which previously lacked scratch implementations.

Task	Best Config	Test Metric	Time (s)
Anomaly Employee	scratch_novel_feature_cross_128x2	0.8347	311.4
Anomaly Heart	scratch_novel_feature_cross_128x2	0.9415	8.8
Anomaly Wine	scratch_novel_feature_cross_128x2	0.9992	55.4
Heart Classification	scratch_novel_feature_cross_128x2	0.9423	7.2
Multi Output Regression	scratch_novel_feature_cross_128x2	0.3504	151.0
Temperature Regression	scratch_novel_feature_cross_128x2	0.9995	126.8
Weather Classification	scratch_novel_feature_cross_128x2	0.5120	192.9
Wind Forecasting	scratch_novel_feature_cross_128x2	0.9597	117.6

The results demonstrate that our novel gating mechanism is highly competitive across all contexts. In anomaly detection (Wine Type and Heart Disease), it achieved near-perfect scores (0.9992 F1 for Wine, 0.9415 ACC for Heart). For regression tasks, it flawlessly matched the temporal dynamics of Temperature (0.9995 R^2) and Wind Forecasting (0.9597 R^2). This confirms that moving beyond standard MLPs to custom topological designs built from primitives can yield highly performant models for tabular data.

9 Conclusion

After running 15 sklearn DNN configs per task across all 10 dashboard tasks, the final score is ML 6 – DNN 4. The ML models dominate for the core supervised tasks (classification, regression, forecasting), mainly because the datasets are tabular and moderate-sized, and the ML baselines were already well-tuned stacking ensembles.

The DNN wins on anomaly detection tasks are mostly explained by the supervised vs unsupervised gap rather than model architecture. The COVID win is genuine but marginal.

The PyTorch follow-up experiments extended this picture across three directions. First, the **standard 10 PyTorch configs** on heart and temperature showed that AdamW (decoupled weight decay) is the single most impactful change over sklearn, gaining $\sim 0.5\text{--}0.7\%$ on both tasks. Second, the same 10 configs on **wind turbine forecasting** confirmed that for smooth time-series regression the PyTorch baseline already beats the sklearn DNN ($+0.04\%$) and nearly matches ML Ridge (-0.04%), with dropout and BatchNorm hurting in this setting. Third, the **6 advanced architecture configs** (weighted loss, wider networks, ResidualMLP skip connections, LR warmup) showed that the wide ResidualMLP (512-dim, 4 blocks) consistently outperforms all MLP-family architectures, adding a further $+0.32\%$ on heart and $+0.34\%$ on temperature over the AdamW baseline.

Finally, we added a **primitive-layer from-scratch implementation** (custom Linear and BatchNorm with `nn.Parameter`) and ran it on heart, temperature, and wind. These scratch models were competitive (and nearly tied the best wind score), confirming that our implementation is not just framework-level wiring, while still showing that architecture choice (wide ResidualMLP) is the main driver of best results on heart and temperature.

Despite these systematic improvements, the ML stacking ensembles remain $\sim 2.5\text{--}3\%$ ahead on the hard tasks (heart, temperature). The bottleneck confirmed by all experiments is not the training framework, the optimiser, or the network depth — it is the fundamental advantage of tree ensembles for tabular data: native handling of categorical variables, feature interaction splits that don't require scaling, and the diversity of multi-model stacking. If we wanted to seriously close the remaining gap, the natural next step would be purpose-built tabular DNN architectures such as TabNet or FT-Transformer, which encode categorical embeddings directly and model feature interactions without requiring hand-crafted preprocessing.

Data sources: `dnn_results/ml_vs_dnn_comparison.csv`, `dnn_results/hp_search/*.csv`, `pytorch_results/pytorch_results/temperature_pytorch_results.csv`, `pytorch_results/pytorch_results/wind_pytorch_results.csv`, `pytorch_results/heart_advanced_results.csv`, `pytorch_results/temperature_advanced_results.csv`, `pytorch_results/scratch_layers_heart_results.csv`, `pytorch_results/scratch_layers_temperature_results.csv`, `pytorch_results/scratch_layers_wind_results.csv`, `pytorch_results/scratch_vs_pytorch_comparison.csv`, `pytorch_results/extended_summary.csv`.